Correcciones Generales de Estilo y Buenas Prácticas Java

Este documento recoge los errores de estilo y buenas prácticas más comunes aplicables a cualquier proyecto Java. Úsalo como referencia de corrección independientemente del dominio o estructura del proyecto.

1. Nombrado (Naming Conventions)

1.1 Paquetes

- **Regla:** solo letras **minúsculas** y números. No se permiten mayúsculas ni guiones bajos (_).
- **Patrón requerido:** `^[a-z]+(\.[a-z][a-z0-9]*)*$`
- **Incorrecto:**

```
package com.empresa.MiProyecto_adaptador;
package com.empresa.app.Model.Entity;
```

- **Correcto:**

```
package com.empresa.miproyecto.adaptador;
package com.empresa.app.model.entity;
```

1.2 Clases e Interfaces

- **Regla:** `UpperCamelCase` (primera letra de cada palabra en mayúscula).
- **Incorrecto:** `productocontrolador` , `producto_controlador`
- **Correcto:** `ProductoControlador` , `IProductoController`

1.3 Métodos

- **Regla:** `lowerCamelCase` . El nombre debe comenzar con al menos dos letras minúsculas antes de usar mayúsculas o guiones bajos.
- **Patrón requerido:** `^[a-z][a-z0-9][a-zA-Z0-9_]*$`
- **Incorrecto:** `ObtenerProducto()` , `obtener_producto()`
- **Correcto:** `obtenerProducto()` , `agregarProductoCarrito()`

1.4 Variables locales y parámetros

- **Regla:** `lowerCamelCase` .
- **Incorrecto:** `String NombreUsuario` , `int total_precio`

- **Correcto:** `String nombreUsuario` , `int totalPrecio`

1.5 Constantes

- **Regla:** `UPPER_SNAKE_CASE` (todas las letras en mayúscula, palabras separadas por `_`).
- **Incorrecto:** `final int maxIntentos = 3;`
- **Correcto:** `static final int MAX_INTENTOS = 3;`

1.6 Campos estáticos (no finales)

- **Regla:** `lowerCamelCase` .
- **Correcto:** `private static int contadorInstancias;`

2. Indentación y Formato

2.1 Espacios vs. Tabuladores

- **Regla:** usar **espacios**, nunca tabuladores (`\t`).
- Configuración `.editorconfig` :
  - Archivos Java: **4 espacios** por nivel de indentación.
  - Resto de archivos: 2 espacios.

2.2 Longitud de línea

- **Máximo:** 120 caracteres por línea.
- Partir líneas largas usando saltos lógicos (después de coma, antes de operador, etc.).
- **Incorrecto:**

```
public List<ProductoDTO> buscarProductosPorCategoria(Long idCategoria, String filtro, boolean ordenado) throw
```

- **Correcto:**

```
public List<ProductoDTO> buscarProductosPorCategoria(
        Long idCategoria, String filtro, boolean ordenado)
        throws ServiceException {
```

2.3 Espacios en blanco

- **No** dejar espacios en blanco al final de una línea (trailing whitespace).
- **No** dejar más de una línea en blanco consecutiva dentro de un bloque.
- Debe haber una línea en blanco entre métodos, campos, constructores, etc.
- El archivo debe terminar con **exactamente una** línea nueva al final.

2.4 Una sentencia por línea

- **Incorrecto:** `int a = 1; int b = 2;`
- **Correcto:**

```
int a = 1;
int b = 2;
```

3. Imports

3.1 Sin imports redundantes ni sin usar

- Eliminar cualquier import que no se use en el archivo.
- No duplicar imports.

3.2 No más de una línea en blanco antes del primer import

- **Incorrecto:**

```
package com.empresa.app;


import java.util.List;
```

- **Correcto:**

```
package com.empresa.app;

import java.util.List;
```

4. Javadoc

4.1 Javadoc en métodos públicos y protegidos

- Los métodos públicos y protegidos con más de 2 líneas deben tener Javadoc.
- Excluidos: métodos anotados con `@Override`, `@Test`, `@BeforeEach`, etc.
- **Correcto:**

```java
/**
 * Busca todos los productos disponibles en una categoría.
 *
 * @param idCategoria identificador de la categoría
 * @return lista de DTOs de producto
 * @throws ServiceException si la categoría no existe
 */
public List<ProductoDTO> buscarPorCategoria(Long idCategoria)
        throws ServiceException {
```

4.2 Javadoc en campos protegidos

- Los campos con visibilidad `protected` necesitan Javadoc.
- Los campos `private` no requieren Javadoc obligatorio.

4.3 Estilo Javadoc

- La primera frase debe terminar en punto.
- No dejar descripciones vacías (`/** */`).

5. Diseño de Clases

5.1 Visibilidad de campos

- Los campos deben ser `private` por defecto; exponer solo mediante getters/setters.
- Evitar campos con visibilidad `package` o `protected` salvo justificación.

5.2 Una clase de nivel superior por archivo

- Cada archivo `.java` debe contener exactamente una clase pública de nivel superior, y su nombre debe coincidir con el del archivo.

6. Números Mágicos

- No usar literales numéricos directamente en el código (salvo `-1` , `0` , `1` y valores en declaraciones de campos).
- **Incorrecto:**

  ```java
  if (precio > 9999L) { ... }
  ```

- **Correcto:**

```java
private static final long PRECIO_MAXIMO = 9999L;
// ...
if (precio > PRECIO_MAXIMO) { ... }
```

7. Orden de Declaraciones en una Clase

Seguir este orden dentro de una clase:

1. Variables estáticas (public → protected → package → private)
2. Variables de instancia (public → protected → package → private)
3. Constructores
4. Métodos (en el mismo orden de visibilidad; agrupar sobrecargas juntas)

8. Orden de Modificadores

El orden correcto de los modificadores es:

```
public protected private abstract default static final transient volatile synchronized native strictfp
```

- **Incorrecto:** `static public final int MAX = 5;`
- **Correcto:** `public static final int MAX = 5;`

9. Bloques y Control de Flujo

9.1 Switch sin `default`

- Todo bloque `switch` debe incluir una cláusula `default`.
- **Correcto:**

```java
switch (estado) {
    case ACTIVO:
        activar();
        break;
    default:
        throw new IllegalStateException("Estado desconocido: " + estado);
}
```

9.2 `default` al final del switch

- La cláusula `default` debe ser la **última** en el bloque `switch`.

9.3 Fall-through en switch

- Si una rama `case` no tiene `break` intencionalmente, documentarlo con un comentario `// falls through`.

9.4 Evitar bloques anidados innecesarios

- No crear bloques `{}` vacíos o anidados sin propósito.

10. Comparaciones y Booleanos

10.1 Comparación de Strings

- Usar `.equals()` en lugar de `==` para comparar objetos.
- **Incorrecto:** `if (nombre == "admin")`
- **Correcto:** `if ("admin".equals(nombre))` (el literal primero evita NPE)

10.2 Simplificar expresiones booleanas

- **Incorrecto:** `if (esValido == true)` / `return (x > 0) ? true : false;`
- **Correcto:** `if (esValido)` / `return x > 0;`

10.3 `equals()` y `hashCode()` van juntos

- Si se sobreescribe uno de los dos métodos, **siempre** sobreescribir ambos.

11. Excepciones

11.1 No capturar excepciones genéricas

- **Incorrecto:** `catch (Exception e)` o `catch (RuntimeException e)`
- **Correcto:** capturar el tipo específico de excepción.

11.2 No lanzar excepciones genéricas

- **Incorrecto:** `throws Exception`
- **Correcto:** `throws ServiceException`, `throws IllegalArgumentException`

12. Otros

12.1 Asignación dentro de expresiones

- No hacer asignaciones dentro de condiciones de `if`, `while`, etc.
- **Incorrecto:** `if ((line = reader.readLine()) != null)`

12.2 No modificar la variable de control de un `for`

- La variable de iteración de un bucle `for` no debe modificarse dentro del cuerpo.

12.3 No usar `clone()` ni finalizadores

- No sobreescribir `clone()` ni `finalize()`.

12.4 No inicializar variables de instancia con su valor por defecto

- **Incorrecto:** `private int contador = 0;` / `private Object obj = null;`
- **Correcto:** `private int contador;` / `private Object obj;`

12.5 Estilo de arrays

- **Incorrecto:** `String args[]`
- **Correcto:** `String[] args`

12.6 Usar `L` mayúscula para literales `long`

- **Incorrecto:** `long valor = 9999l;` (la `l` minúscula se confunde con `1`)
- **Correcto:** `long valor = 9999L;`

12.7 Paréntesis innecesarios

- Evitar paréntesis redundantes que no aporten claridad.
- **Incorrecto:** `return (x + y);`
- **Correcto:** `return x + y;`

Plantilla de Corrección por Alumno

> Copia esta sección para cada alumno que vayas a evaluar.

Alumno: Alex e Ivan
Proyecto: PROG_Proyecto_Steam
Fecha: 2026-02-26

#	Sección	Comprobación	Resultado	Observaciones
1.1	Nombrado	Paquetes en minúsculas sin mayúsculas ni `_`	✅	Todos los paquetes (`org.alexyivan` , `org.alexyivan.controlador` , etc.) son correctos
1.2	Nombrado	Clases e interfaces en `UpperCamelCase`	❌	Enums usan sufijo `ENUM` en mayúsculas (`EstadoCompraENUM` , `PegiENUM` …); DTOs usan `DTO` en mayúsculas (`BibliotecaDTO` , `JuegoDTO` …)
1.3	Nombrado	Métodos en `lowerCamelCase`	✅	Todos los métodos revisados siguen el patrón correcto
1.4	Nombrado	Variables locales y parámetros en `lowerCamelCase`	❌	Parámetros `usuarioID` y `juegoID` en `IBibliotecaControlador` / `BibliotecaControlador` (deberían ser `usuarioId` , `juegoId`)
1.5	Nombrado	Constantes en `UPPER_SNAKE_CASE`	❌	`JuegoControlador` : `private final int DESCUENTO_MIN` y `DESCUENTO_MAX` no son `static` ; los campos de instancia `final` no son constantes y no deben usar UPPER_SNAKE_CASE
1.6	Nombrado	Campos estáticos no finales en `lowerCamelCase`	✅	`idCounter` , `juegos` , `usuarios` , `listaPaises` , etc. correctos
2.1	Formato	Indentación con 4 espacios (sin tabuladores)	❌	`JuegoRepoInMemory.java` (método `actualizar`): bloque sin indentación dentro del cuerpo del método
2.2	Formato	Líneas de máximo 120 caracteres	❌	`EstadisticasJuegoDTO.java` : constructor en una sola línea con 11 parámetros supera 120 chars; también `CompraForm.java` y `UsuarioRepoInMemory.java`
2.3	Formato	Sin espacios en blanco al final de línea; archivo termina con una línea nueva	❌	`EstadisticasJuegoDTO.java` y `EstadoInstalacionENUM.java` terminan con dos saltos de línea en blanco
2.4	Formato	Una sentencia por línea	❌	`BibliotecaForm.java` : `this.ultimaFechaJuego = ultimaFechaJuego;;` — doble punto y coma, genera sentencia vacía adicional
3.1	Imports	Sin imports no utilizados ni duplicados	❌	`CompraControlador.java` (`IBibliotecaRepo`), `ResenhaControlador.java` (`IJuegoRepo`),

#	Sección	Comprobación	Resultado	Observaciones
				`EstadisiticasJuegoEntidad.java` (`JuegoDTO` , `UsuarioDTO` , `Optional`), `IBibliotecaControlador.java` (`JuegoDTO` , `Optional`), `IUsuarioControlador.java` (`LocalDate`), `IUsuarioRepo.java` (`ValidacionException` , `UsuarioDTO`)
3.2	Imports	Solo una línea en blanco entre `package` e imports	✅	Correcto en todos los archivos
4.1	Javadoc	Javadoc en métodos públicos y protegidos (>2 líneas)	❌	Sin Javadoc en todos los métodos `validar()` de los `*Form.java` , todos los métodos de `Mapper.java` , y las interfaces `IBibliotecaControlador` , `IJuegoControlador` , `IUsuarioControlador` , etc.
4.2	Javadoc	Javadoc en campos `protected`	✅	No hay campos `protected` en el proyecto
4.3	Javadoc	Primera frase del Javadoc termina en punto; sin Javadoc vacíos	✅	El único Javadoc existente (`ICrud.java`) es correcto
5.1	Diseño	Campos `private` expuestos solo con getters/setters	❌	`UsuarioForm.java` : 10 campos de instancia sin modificador (package-private). `BibliotecaDTO.java` : campo `fechaAdquisicion` package-private. `UsuarioRepoInMemory.java` : `public static List<String> listaPaises` (debería ser `private`)
5.2	Diseño	Una clase pública de nivel superior por archivo	✅	Cada `.java` contiene exactamente un tipo público
6	Números mágicos	Sin literales numéricos directos en el código (usar constantes)	❌	`UsuarioForm` (`minusYears(13)`), `BibliotecaForm` (`tiempoJuegoTotal * 10`), `BusquedaJuegosForm` (`.length() > 2`)
7	Orden declaraciones	Orden correcto: estáticas → instancia → constructores → métodos	✅	Correcto en todos los archivos revisados
8	Modificadores	Modificadores en el orden correcto (`public static final...`)	✅	Correcto en todos los archivos

#	Sección	Comprobación	Resultado	Observaciones
9.1	Control de flujo	Llaves `{}` en todas las estructuras de control	❌	`JuegoControlador.java` y `UsuarioControlador.java` : `if (!errores.isEmpty()) throw ...` sin llaves. `Mapper.java` : `if (entidad == null) return null;` sin llaves en todos los métodos
9.2	Control de flujo	`switch` incluye cláusula `default` al final	✅	No hay sentencias `switch` en el código
9.3	Control de flujo	Fall-through en `switch` documentado con `// falls through`	✅	No hay sentencias `switch` en el código
9.4	Control de flujo	Sin bloques `{}` anidados o vacíos innecesarios	✅	No se encontraron bloques innecesarios
10.1	Comparaciones	Comparar objetos con `.equals()` , no con `==`	✅	No se encontraron comparaciones `==` sobre objetos
10.2	Comparaciones	Expresiones booleanas simplificadas	✅	No hay patrones `== true` / `== false` ni ternarios redundantes
10.3	Comparaciones	`equals()` y `hashCode()` siempre sobreescritos juntos	✅	Ninguna clase sobreescribe `equals()` ni `hashCode()`
11.1	Excepciones	No capturar excepciones genéricas (`Exception` , `RuntimeException`)	✅	No hay bloques `catch` en el código
11.2	Excepciones	No lanzar `throws Exception` genérico	✅	Solo se declara `throws ValidacionException` (excepción específica)
12.1	Otros	Sin asignaciones dentro de condiciones (`if` , `while` ...)	✅	No se encontraron
12.2	Otros	No modificar la variable de control de un `for`	✅	Solo se usan Streams; no hay bucles `for` clásicos
12.3	Otros	No sobreescribir `clone()` ni `finalize()`	✅	Ninguna clase sobreescribe estos métodos
12.4	Otros	No inicializar campos de instancia con su valor por defecto	✅	Ningún campo se inicializa explícitamente a `0` , `null` o `false`

#	Sección	Comprobación	Resultado	Observaciones
12.5	Otros	Arrays declarados como `String[] args`, no `String args[]`	✅	`App.java` usa `String[] args` correctamente
12.6	Otros	Literales `long` con `L` mayúscula (`9999L`, no `9999l`)	✅	No se encontraron literales `long` en el código
12.7	Otros	Sin paréntesis innecesarios	✅	No se encontraron paréntesis redundantes

Checks superados: 24 / 36

Comentarios generales:

> El proyecto muestra una estructura clara y bien organizada (paquetes, separación en capas, uso de Streams). Los fallos más relevantes son:
>
> - **Nomenclatura masiva**: todos los enums (`*ENUM`) y todos los DTOs (`*DTO`) violan la convención UpperCamelCase para los sufijos (`Enum`, `Dto`).
> - **Javadoc ausente**: ningún método de negocio, de repositorio ni de interfaz tiene documentación. Es la deficiencia más extensa del proyecto.
> - **Visibilidad de campos**: `UsuarioForm` expone sus 10 campos sin modificador de acceso; `UsuarioRepoInMemory.listaPaises` es público.
> - **Llaves en control de flujo**: varios `if` de una línea en controladores y `Mapper` omiten las llaves obligatorias.
> - **Números mágicos** puntuales (`13`, `10`, `2`) que deberían definirse como constantes nombradas.